

Rapport de sécurité

Hermanas — système d'automatisation du poulailler

Date : 02 June 2026

Version analysée : 0.8.1 (branche *master*)

Périmètre : backend Spring Boot 2.7.18 (Java 11) + frontend Angular 20

Déploiement : Raspberry Pi Zero, exposé publiquement sur www.hermanas.fr derrière un
reverse-proxy Caddy

Document interne — ne pas diffuser à l'extérieur de l'équipe technique.

1. Résumé exécutif

Hermanas est une application IoT personnelle : elle automatise un poulailler (porte motorisée, éclairage, ventilation, caméra) à partir d'un Raspberry Pi Zero. Bien que le périmètre fonctionnel soit étroit, l'application est exposée publiquement sur Internet, ce qui justifie une posture de sécurité solide.

L'audit confirme que les fondamentaux sont en place : authentification BCrypt, autorisation par rôle explicite, CSRF activé, masquage des données sensibles dans les réponses, journal d'événements pour l'audit. Les principaux risques résiduels sont structurels (stack EOL imposée par le matériel Pi Zero) ou opérationnels (secrets par défaut à override, en-têtes de sécurité délégués au reverse-proxy).

Posture globale

Axe	Niveau	Commentaire
Authentification	Bon	BCrypt, session + remember-me, état pending validation
Autorisation	Bon	Rôles explicites, @PreAuthorize + règles par chemin
CSRF / cookies	Bon	Token CSRF activé, échange via header
En-têtes HTTP	Moyen	HSTS/CSP absents — délégués au reverse-proxy
Validation entrée	Moyen	DTOs + path-traversal OK ; bornes manquantes sur /config
Rate limiting	Faible	Présent sur shutdown/reboot, absent du login
Secrets	Moyen	Clés par défaut à override, pas de validation
Audit	Moyen	Bon sur porte/auth, partiel sur config
Dépendances	Moyen	Spring Boot 2.7 EOL, contrainte matérielle
Exposition réseau	Bon	Actuator restreint, photo archive privée

Recommandation générale. L'application reste adaptée à son usage actuel — domestique, à faible enjeu — sous réserve que le reverse-proxy Caddy applique bien HSTS et HTTPS strict. Trois actions sont prioritaires : ajouter un rate-limit au login, externaliser systématiquement la clé remember-me, et planifier la migration du Pi Zero vers un matériel supportant Java 17 (donc Spring Boot 3.x) pour retrouver l'accès aux correctifs de sécurité.

2. Périmètre et méthode

2.1 Architecture analysée

Composant	Version	Localisation
Backend Spring Boot	2.7.18	pom.xml
JDK	11	Contrainte Pi Zero
Spring Security	5.7.x (bundled)	—
MariaDB	—	Production
H2 (tests)	—	Profil gpio-fake
Frontend Angular	20.x	frontend/package.json
WebSocket (STOMP)	—	websocket/

Composant	Version	Localisation
Reverse proxy	Caddy	Hors dépôt

2.2 Méthode

Revue statique du code source sur la branche *master* au 02 June 2026. Inspection des contrôleurs REST, de la configuration Spring Security, des annotations `@PreAuthorize`, des entités JPA, du `application.properties`, des règles de routage Angular et des gardes. Pas de scan dynamique ni de test d'intrusion : ce rapport reflète l'état du code, pas l'état du déploiement.

2.3 Légende des niveaux de risque

Niveau	Définition
Critique	Exploit direct possible, impact majeur
Élevé	Exploit conditionnel ou impact significatif
Moyen	Risque réel mais limité par d'autres contrôles
Faible	Best-practice non respectée, impact résiduel
Info	Observation neutre / décision documentée

3. Authentication

3.1 Points forts

- **Hachage BCrypt** — `BCryptPasswordEncoder` déclaré en bean (*SecurityConfig.java:228*). Algorithme standard, salage automatique, coût par défaut (10).
- **Stockage en base** — `DbUserDetailsService` lit les comptes depuis la table `users`. Bootstrap depuis `users.properties` au premier démarrage, puis table de référence.
- **État compte « non validé »** — rôle `NOT_VALIDATED_YET` et `PendingValidationUserDetailsService` empêchent la connexion tant qu'un administrateur n'a pas promu le compte. Évite l'auto-enrôlement public.
- **Session + remember-me persistant** — cookie session `JSESSIONID` ; cookie `remember-me` `hermanas-remember-me` avec tokens stockés en base (`persistent_logins`, rotation à chaque réutilisation). Validité 31 jours (*application.properties:205*).
- **Journalisation des connexions** — succès et échecs enregistrés avec login tenté, raison et IP cliente (*SecurityConfig.java:167-190*). Permet la détection a posteriori de brute-force depuis la page Journalisation.

3.2 Risques

Risque	Sévérité	Référence
Pas de rate-limit sur <code>/api/v1/auth/login</code> — brute-force possible (élevé par BCrypt par défaut)	Élevé	<i>SecurityConfig.java:190</i>
Pas de rate-limit sur <code>/api/v1/auth/register</code> — possibilité de remplir la table <code>users</code> avec des comptes non validés	Moyen	<i>SecurityConfig.java:191</i>
Cookie <code>remember-me</code> non secure par défaut (<code>setUseSecureCookie(false)</code>) — réinitialisé en prod si Caddy force HTTPS	Moyen	<i>SecurityConfig.java:280</i>
Réponse 401 différenciée pour comptes en attente de validation (erreur <code>SECURITY_CONFIG_INVALIDATION</code>) — révèle l'existence de comptes non validés	Faible	<i>SecurityConfig.java:183-190</i>

Risque	Sévérité	Référence
Clé remember-me par défaut « change-me-in-production-please » dans application.properties si 200	Élevée	application.properties si 200

3.3 Recommandations

- Ajouter `@RateLimited` sur `/api/v1/auth/login` et `/api/v1/auth/register` (mécanisme déjà disponible dans `security/ratelimit/`). Cible : 5 tentatives par IP et par minute.
- Vérifier en démarrage que la clé remember-me a été surchargée (échec rapide si la valeur par défaut est détectée). Au minimum, un log `WARN` au boot.
- Aligner le code 401 du compte en attente de validation sur le 401 générique et différencier le message uniquement après la première connexion réussie.

4. Autorisation

4.1 Modèle de rôles

Rôle	Définition
USER	Compte authentifié standard — accès à la caméra (photos), actions sur la porte/lampe/ventilo/musique
ADMIN	Tout USER + administration : reboot/shutdown, <code>/actuator/**</code> , <code>/logs/**</code> , <code>/events/auth/**</code> , gestion utilisateurs
NOT_VALIDATED_YET	Placeholder pour compte fraîchement enregistré — aucune permission, login refusé

4.2 Règles d'autorisation

Backend. Combinaison de règles par chemin (`antMatchers` dans `SecurityConfig#filterChain`) et d'annotations `@PreAuthorize` au niveau méthode. La politique par défaut est *permitAll* ; chaque chemin sensible est explicitement protégé.

Chemin / action	Politique
GET <code>/api/v1/auth/me</code>	<code>permitAll</code>
POST <code>/api/v1/auth/{login,logout,register}</code>	<code>permitAll</code>
GET <code>/api/v1/push/vapid-public-key</code>	<code>permitAll</code>
POST <code>/api/v1/push/test</code>	ADMIN
GET <code>/actuator/health</code> , <code>/actuator/info</code>	<code>permitAll</code>
<code>/actuator/**</code> (autres)	ADMIN
<code>/api/v1/logs/**</code>	ADMIN
<code>/api/v1/events/auth/**</code>	ADMIN
<code>/api/v1/buttons/**</code> , <code>/api/v1/email/**</code>	ADMIN
<code>/api/v1/camera/photos/**</code>	USER ou ADMIN
POST/PUT/DELETE/PATCH <code>/api/v1/**</code>	USER ou ADMIN
GET <code>door/turn*</code> , <code>fan/switch</code> , <code>music/switch</code> , ...	USER ou ADMIN
Reste (GET API + SPA + Swagger + ws)	<code>permitAll</code>

Frontend. Trois mécanismes :

- AuthGuard sur /camera et /notification
- AdminGuard sur /energy
- Conditions @if (isAdmin()) dans les templates (par ex. side-nav, page Système, page Caméra) — défense en profondeur, complément du backend

4.3 Risques

Risque	Sévérité	Référence
Pas de hiérarchie de rôles — un ADMIN ne récupère pas USE	FAIBLE	SecurityConfig.js:1134 et hasAnyRole pour comp
Promotion forcée du compte « marguerite » en ADMIN à chaq	FAIBLE	DeploymentService.java:7780
Aucune protection contre l'IDOR sur GET /api/v1/residents/{id} (à confirmer)	FAIBLE	ResidentsController

5. CSRF, CORS et en-têtes HTTP

5.1 CSRF

CSRF est **activé** via `CookieCsrfTokenRepository.withHttpOnlyFalse()` (*SecurityConfig.java:85*).
Le token est exposé dans le cookie XSRF-TOKEN ; Angular l'ajoute automatiquement à chaque requête mutante dans l'en-tête X-XSRF-TOKEN. Schéma idiomatique pour une SPA même-origine.

5.2 CORS

Aucune origine CORS n'est autorisée par défaut (`cors.origins.allowed` vide, *application.properties:217*). Le SPA Angular étant bundlé dans le JAR et servi same-origin, ce choix est correct. Le canal WebSocket STOMP applique la même configuration (*WebSocketConfig.java*).

5.3 En-têtes de sécurité

En-tête	Statut	Commentaire
X-Frame-Options	Désactivé volontairement	frameOptions().disable() — permet l'embarquement de cartes/charts ; ex
X-Content-Type-Options	Non configuré côté app	Souvent appliqué par Caddy
Strict-Transport-Security (HSTS)	Non configuré côté app	Doit être ajouté côté Caddy
Content-Security-Policy	Non configuré	Image distante (board Pi) charge depuis othermod.com ; CSP minimalis
Referrer-Policy	Non configuré	À ajouter côté Caddy ou via WebMvcConfigurer

5.4 Recommendations

- Documenter dans le README la configuration Caddy attendue (HSTS, X-Content-Type-Options, Referrer-Policy, redirection 80→443).
- Définir une CSP minimaliste au niveau Spring (SecurityFilterChain  contentSecurityPolicy("default-src 'self'; img-src 'self' https://othermod.com data:; ...")) pour ne pas dépendre uniquement de Caddy.
- Réactiver X-Frame-Options sur les endpoints non concernés par l'embarquement, ou passer en SAMEORIGIN.

6. Validation des entrées

6.1 Points forts

- **Bean Validation** appliquée sur les contrôleurs critiques : `@Valid` `@RequestBody` sur `ResidentRequest`, `UserCreateRequest` ; `@Min/@Max` sur les calibrations servo et le nombre de lignes de logs.
- **GlobalExceptionHandler** convertit `MethodArgumentNotValidException` et `ConstraintViolationException` en JSON structuré (400 Bad Request) sans fuiter la stack-trace.
- **Upload de fichiers** : limite 5 Mo par fichier (`spring.servlet.multipart.max-file-size = 5MB`, `application.properties:193`). MIME whitelist {jpeg, png, webp} dans `ResidentPhotoStorage`. Redimensionnement client à ~300 ko avant envoi.
- **Protection path traversal** centralisée : `ResidentPhotoStorage.resolve()` et `LogsService.resolveSafe()` normalisent puis vérifient `startsWith` sur le répertoire racine.
- **Pas d'injection SQL** détectée — usage exclusif de Spring Data JPA avec requêtes paramétrées.

6.2 Risques

Risque	Sévérité	Référence
Endpoints <code>/api/v1/config/**</code> acceptent des paramètres <code>@RequestBody</code> sans bon <code>Content-Type</code> (ex. jpeg), positions servo hors plage.	Moyenne	<code>ConfigRestController.java:57-60</code>
MIME validation basée sur le header <code>Content-Type</code> de la requête. Facile à falsifier.	Faible	<code>ResidentPhotoStorage.java:57-60</code>
Le viewer de logs autorise tout fichier matchant des patterns. Facile à répéter le répertoire racine.	Critique	<code>LogsService.java:46-52</code>

6.3 Recommandations

- Ajouter `@Min/@Max` sur tous les setters de `ConfigRestController` (port 1-65535, durées 1-60000 ms, positions servo 1-100, etc.).
- Compléter la validation upload par une lecture des magic bytes (au moins JPEG SOI 0xFFD8FFE0).
- Restreindre le viewer de logs à une whitelist explicite de noms (`spring.log` et ses rotations seulement).

7. Exposition des données

7.1 Données sensibles masquées

- **Mot de passe SMTP** — exposé via `password_set: true|false`, jamais la valeur (`ConfigRestController.java:190`).
- **Clé API OpenWeather** — exposée via `key_set + key_length`, jamais la valeur (`ConfigRestController.java:170-172`).
- **Coordonnées GPS** du poulailler — non retournées dans `GET /api/v1/config`, modifiables uniquement via les endpoints PUT dédiés (`location/latitude` et `location/longitude`). Décision explicite documentée dans `WeatherSettings`.
- **DTOs en frontière** — `ResidentDTO`, `UserDTO`, `SensorDTO`, `EventDTO`. Les entités JPA ne sont jamais retournées telles quelles.
- **Stack-traces** — masquées par `GlobalExceptionHandler` (500 générique côté client, détails uniquement dans les logs serveur).

7.2 Risques résiduels

Risque	Sévérité	Référence
Le journal métier (/api/v1/events/business) est public et expose l'historique des événements des résidentes, etc. — permet de savoir si une résidente a été créée, etc.	Moyenne	SecurityConfig.java, EventsRestController, etc.
/actuator/health est public et révèle l'état des composants (disque plein, capteur de température, etc.)	Faible	HealthConfig.java, 104905
Filename des photos résidentes retourné (UUID désormais), mais la liste complète des résidentes est lisible par tout authentifié — pas de fuite de données	Faible	ResidentsRestController, 104905

8. Journalisation et audit

8.1 Couverture du journal d'événements

L'énumération EventType couvre les principales actions métier :

Catégorie	Événements
Authentification	LOGIN_SUCCESS, LOGIN_FAILED, LOGOUT
Porte	DOOR_OPENED, DOOR_CLOSED, DOOR_OPEN_FAILED, DOOR_CLOSE_FAILED, DOOR_POSITION_CHANGED
Périphériques	LIGHT_ON/OFF, FAN_ON/OFF, MUSIC_STARTED/STOPPED, COCORICO
Résidentes	RESIDENT_CREATED, RESIDENT_DELETED
Système	STARTUP, SHUTDOWN, SHUTDOWN_REQUESTED, REBOOT_REQUESTED

8.2 Annotation @AuditLog

Une annotation maison @AuditLog(category, operation) permet de tracer les opérations sensibles. Elle est aujourd'hui posée sur les actions porte (ouverture/fermeture) et système (reboot/shutdown).

Couverture incomplète : les changements de configuration (SMTP, GPS, URL d'inférence IA, clé OpenWeather) ne génèrent qu'un log INFO standard.

8.3 Risques

Risque	Sévérité	Référence
Modifications de configuration non auditées explicitement — un attaquant peut changer les paramètres de l'application	Moyenne	ConfigRestController, SMTP.java, etc.
EventType stocké par ordinal — un réordonnancement de l'énumération EventType peut rendre les logs d'avertissement présents obsolètes	Moyenne	EventType.java, Historique.java
Pas de rotation par taille — le fichier spring.log peut saturer le disque du Pi Zero	Faible	application.properties (logging*)

8.4 Recommandations

- Étendre @AuditLog à tous les setters de ConfigRestController, EnergyRestController, UserRestController (catégories CONFIG, ENERGY, USER).
- Migrer EventType vers une persistance par name() (@Enumerated(EnumType.STRING)) — coût ponctuel : migration de la colonne, gain : robustesse à long terme.
- Activer une rotation par taille (Logback : SizeAndTimeBasedRollingPolicy, 10 Mo / fichier, conservation 7 jours).

9. Gestion des secrets

9.1 État actuel

- **users.properties** exclus du dépôt (*.gitignore* ligne 36) ; chemin configurable via `hermanas.security.users-file` (défaut `./users.properties` à côté du JAR).
- **CLI de génération de hash** intégrée : `java -jar hermanas.jar --hash <password>` — évite de coller le mot de passe en clair dans un terminal externe.
- **Setters d'admin** pour SMTP, OpenWeather, etc. — la valeur écrite est stockée en BDD via Parameter et n'apparaît jamais dans une réponse REST.

9.2 Risques

Risque	Sévérité	Référence
Clé remember-me « change-me-in-production-please » par défaut sans validation de l'application	Élevée	application.properties 203
Placeholders de configuration SMTP et clé OpenWeather exposés en clair dans application.properties (valeurs sentinelles), à override	Élevée	application.properties 24, 142-146
Pas d'intégration coffre-fort (Vault, AWS Secrets Manager, Spring Cloud Config) — les secrets de prod vivent dans /home/pi/application	Info	Le secret de prod

9.3 Recommandations

- Bloquer le démarrage si `hermanas.security.remember-me-key` est la valeur par défaut (échec rapide explicite, ou minimum log WARN).
- Sortir `spring.mail.password` de `application.properties` : ne plus laisser de placeholder en clair, n'accepter que l'override BDD ou variable d'environnement.
- Tourner la clé remember-me en cas de fuite supposée (invalide tous les tokens persistant_logins existants — comportement intentionnel).

10. Rate-limiting et abuse

Une implémentation maison (package `security/ratelimit/`) propose une annotation `@RateLimited(maxRequests, perSeconds)` appliquée par un aspect. Le compteur est en mémoire (par IP), reset au redémarrage — adapté au Pi Zero (instance unique).

Couverture actuelle

Endpoint	Limite
POST /api/v1/system/shutdown	2 requêtes / 5 min / IP
POST /api/v1/system/reboot	2 requêtes / 5 min / IP
POST /api/v1/auth/login	Aucune
POST /api/v1/auth/register	Aucune
Reste	Aucune

Recommandations

- Étendre la limitation à `/api/v1/auth/login` (5 essais / minute / IP).
- Étendre à `/api/v1/auth/register` (3 enregistrements / heure / IP).
- Ajouter un bandeau d'avertissement et un log AUDIT lorsqu'un seuil est atteint.

- Documenter le comportement attendu derrière reverse-proxy (clé du compteur = X-Forwarded-For, déjà géré côté code).

11. Dépendances et chaîne d'approvisionnement

11.1 Backend

Composant	Version	Statut
Java	11	LTS — fin de support septembre 2026
Spring Boot	2.7.18	EOL novembre 2023 — bloqué par Pi Zero
Spring Security	5.7.x (bundled)	EOL avec Boot 2.7
@EnableGlobalMethodSecurity	Déprécié	Remplacement requiert Boot 3.x
javax.* namespaces	Legacy	Migration jakarta.* impossible sur Boot 2.7
Pi4j	≤ 2.4.0	Pin matériel — versions ultérieures incompatibles Pi Zero+Java 11
MariaDB Connector/J	3.5.8	À jour
BouncyCastle	1.70	Récent
commons-io	2.18.0	Récent

11.2 Frontend

Composant	Version	Statut
Angular	20.x	Récent, supporté
RxJS	7.8.1	Récent
Bootstrap	5.3.3	Récent
aws-amplify et libs associées	Retirées	Migration phase 7 — 310 paquets désinstallés

11.3 Risque structurel

Le stack backend est dans une fenêtre de risque qui ne se résoudra qu'avec un upgrade matériel. Tant que le Pi Zero est en service :

- Pas de patches de sécurité officiels Spring Boot 2.7 — une CVE Boot/Security future ne sera pas corrigée par l'éditeur.
- Surveillance manuelle requise (NVD, advisories Spring, snyk advisor).
- Pas de migration jakarta.* possible — toute dépendance qui basculera vers Jakarta EE 9+ deviendra inutilisable sans rétro-portage.

11.4 Recommandations

- Lancer périodiquement `mvn dependency-check:check` (OWASP Dependency-Check, gratuit) ; configurer un seuil d'alerte à CVSS ≥ 7.
- Planifier l'upgrade matériel vers un Raspberry Pi 4/5 pour rester sur Boot 3.x. Échéance recommandée : avant le 31 décembre 2026.
- Pour le frontend : `npm audit --production` en CI.

12. Opérationnel et réseau

12.1 Endpoints d'administration

Les endpoints Actuator sont restreints à un sous-ensemble : `health`, `info`, `metrics`, `loggers`, `prometheus` (`application.properties:22`). Tous les endpoints potentiellement dangereux sont explicitement désactivés :

```
management.endpoint.heapdump.enabled=false
management.endpoint.threaddump.enabled=false      management.endpoint.env.enabled=false
management.endpoint.configprops.enabled=false    management.endpoint.beans.enabled=false
management.endpoint.shutdown.enabled=false
```

`/actuator/health` et `/actuator/info` sont publics ; le reste exige le rôle ADMIN.

12.2 HTTPS et reverse-proxy

Le code suppose un déploiement derrière Caddy : il fait confiance à X-Forwarded-For pour la journalisation IP (`SecurityConfig.java:217`) et désactive le flag `secure` du cookie `remember-me` pour permettre le dev HTTP. **La configuration Caddy n'est pas dans ce dépôt** et doit être documentée à part : redirection 80→443, HSTS, headers de sécurité, blocage des en-têtes X-Forwarded-* en entrée pour éviter le spoofing.

12.3 Système de fichiers

Chemin	Contenu	Permissions
<code>./residents-photos/</code>	Photos uploadées	Hérité de l'umask Linux
<code>./log/</code>	Logs applicatifs	Hérité de l'umask Linux
<code>./photos/</code>	Captures caméra	Hérité de l'umask Linux
<code>./users.properties</code>	Hashes bcrypt (bootstrap)	À sécuriser (chmod 600)
<code>./application.properties</code>	Surcharges runtime	À sécuriser (chmod 600)

12.4 Exécution de commandes externes

L'application invoque des binaires shell pour la caméra (`camera.streaming.command`), le lecteur audio (`music.player.start.cmd`) et le mixer (`music.volume.cmd`). Les chemins sont définis dans `application.properties` ; ils ne reçoivent **pas** d'input utilisateur, donc pas d'injection de commande côté API. Garder cette discipline : ne jamais concaténer une valeur `@RequestParam` dans une commande shell.

12.5 Recommandations

- Versionner la configuration Caddy à côté du repo (sous-dossier `infra/`) ou la documenter dans le README.
- Appliquer `chmod 600` et `chown pi:pi` sur `users.properties` et la surcharge `application.properties` de la Pi.
- Vérifier dans la configuration Caddy que les en-têtes X-Forwarded-* entrants sont écrasés (`header_up X-Forwarded-For {http.request.remote.host}`).

13. Plan d'actions priorisé

13.1 Priorité 1 — à traiter sous 1 mois

Action	Effort	Risque adressé
Rate-limit sur /auth/login (5/min/IP)	S	Brute-force
Rate-limit sur /auth/register (3/h/IP)	S	Spam comptes
Fail-fast si remember-me-key est la valeur par défaut	XS	Tokens prédictibles
Documenter / versionner la conf Caddy (HSTS, CSP, headers)	S	Headers de sécurité

13.2 Priorité 2 — à traiter sous 3 mois

Action	Effort	Risque adressé
Bornes @Min/@Max sur tous les setters Config	M	Stabilité / abus
@AuditLog sur les setters Config / User	S	Audit incomplet
Migrer EventType vers @Enumerated(String)	M	Robustesse historique
Rotation logs par taille (SizeAndTimeBased)	S	Saturation disque Pi
Magic-byte check sur upload photos	S	Bypass MIME
CSP minimaliste côté Spring	S	Défense en profondeur

13.3 Priorité 3 — stratégique (6-12 mois)

Action	Effort	Risque adressé
Upgrade matériel Pi 4/5 → Boot 3.x + Java 21	XL	Stack EOL
OWASP Dependency-Check en CI	M	Veille CVE
npm audit en CI	S	Veille front
Test d'intrusion externe	L	Validation black-box

13.4 Légende effort

XS < 2 h · S = 1/2 j · M = 1-2 j · L = 1 semaine · XL = projet

14. Conclusion

Hermanas présente une posture de sécurité **raisonnable** au regard de son périmètre et de sa cible (instance unique, usage personnel, exposition publique limitée au site vitrine et au tableau de bord).

Les fondamentaux essentiels — authentification BCrypt, autorisation explicite par rôle, CSRF, masquage des secrets dans les réponses, journal d'événements — sont correctement mis en œuvre. Les faiblesses identifiées sont soit structurelles (stack EOL imposée par le matériel), soit opérationnelles (secrets par défaut, en-têtes délégués à Caddy), soit incrémentales (validation des bornes, audit plus large).

Les trois priorités à court terme — ajouter un rate-limit au login, durcir la clé remember-me et documenter la configuration Caddy — sont à faible coût et fortement amélioratives. La priorité stratégique reste le

renouvellement matériel du Pi Zero pour sortir de la dette Spring Boot 2.7 / Java 11.

Ce rapport reflète l'état du code au 02 June 2026. À renouveler à chaque évolution majeure (rôle, endpoint sensible, mise à jour de dépendance critique) ou au minimum une fois par an.